Les formulaires

Les formulaires introduits dans le HTML depuis ses plus anciennes versions sont l'élément essentiel qui permet l'interactivité entre un site et ses visiteurs. Ils constituent pour cette raison la base de la création de sites dynamiques. L'envoi d'informations du poste client vers le serveur *via* le protocole HTTP concerne aussi bien l'envoi des données personnelles d'un internaute qui souhaite passer une commande que le déclenchement d'une requête dans une base de données ou la création de page dynamique en réponse à cette demande.

Tout échange entre visiteur et serveur passe par un formulaire, dans lequel l'utilisateur peut saisir textes ou mots de passe, opérer des choix grâce à des boutons radio, des cases à cocher ou des listes de sélection, voire envoyer ses propres fichiers depuis le poste client.

Création d'un formulaire HTML

Avant toute chose, il faut créer la structure XHTML d'un formulaire.

Pour être conforme au XHTML 1.1, le document contenant le formulaire doit avoir la structure minimale suivante :

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
 <head>
  <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
  <title>Titre de la page</title> ←Ⓙ
```

```
</head>
<body>
<form method="post" action="nomdefichier.php">
 <fieldset>
   <legend>Titre du formulaire</legend>
   <!— Corps du formulaire contenant les différentes composants—>
 </fieldset> ←@
</form>
</body>
</html>
```

L'élément <form> possède certains attributs obligatoires. D'autres sont utilisés dans des circonstances particulières en fonction des besoins.

L'attribut action="nom_de_fichier.php" est obligatoire. Il désigne le fichier qui va traiter, sur le serveur, les informations saisies dans le formulaire. Il est recom- mandé que ce fichier soit présent dans le même répertoire que celui contenant le formu- laire, mais ce n'est pas obligatoire. Si le fichier se trouve dans un autre dossier, voire sur un autre serveur, il faut utiliser une adresse absolue, en écrivant, par exemple :

```
action= "http://www.abcphp.com/dossier/nom_de_fichier.php"
```

Dans ce cours il s'agira toujours d'un fichier PHP. Il existe d'autres solutions sur des serveurs non-PHP, comme ASP.Net, qui est un concurrent de PHP.

Pour que le fichier qui traite les données soit à coup sûr celui qui contient le formulaire, vous pouvez utiliser la variable $_SERVER["PHP_SELF"], qui contient le nom du fichier en cours d'exécution comme valeur de l'attribut action. Pour les versions antérieures à PHP 4.1, il fallait utiliser la variable $PHP_SELF. En cas de maintenance du code entraînant le changement du nom du fichier, il n'est pas nécessaire de modifier la valeur de l'attribut action.

Vous pouvez avoir, par exemple, le code suivant :

```
<form method="post" action="<?= $_SERVER["PHP_SELF"] ?>">
```

L'attribut action="nom_de_fichier.php" peut aussi avoir la valeur "mailto:", qui provoque l'envoi des données vers une adresse e-mail, qu'il faut préciser à la suite du mot mailto en écrivant, par exemple :

```
action="mailto:nom@abcphp.com"
```

Cette méthode ne peut servir qu'à envoyer des informations vers un mail mais pas vers une base de données. Comme elle ne permet pas de les traiter facilement, elle ne nous est pas très utile en PHP.

method="post|get" détermine la méthode d'envoi des données vers le serveur. La méthode get, qui est la méthode par défaut, présente l'inconvénient d'ajouter les données du formulaire à l'adresse URI du fichier qui les traite, ce qui les rend visibles par le visiteur. Cet inconvénient peut être exploité pour passer des données à un script dès son appel. De plus, il existe une limite à la longueur des URI et donc à la quantité de données à

transmettre. Ces problèmes ne se retrouvent pas avec la valeur post, que vous utiliserez dans la plupart des cas.

name="chaine_de_caractères" attribue un nom au formulaire. Cet attribut est surtout utilisé pour accéder aux éléments du formulaire *via* un script JavaScript.

enctype="type_d'encodage" détermine le type d'encodage des données transmises au serveur. Sa valeur par défaut, "application/x-www-form-urlencoded", est utilisable dans la plupart des cas, à l'exception du transfert de fichiers du poste client vers le serveur, pour lequel elle doit être "multipart/form-data". Si l'attribut action a la valeur "mailto:", l'attribut enctype a pour valeur "text/plain" ou "text/html", selon que le contenu est envoyé à une adresse e-mail au format texte ou XHTML.

L'élément <fieldset> (repère @) permet, à l'intérieur d'un même formulaire, de créer des blocs visuels contenus entre les balises <fieldset> et </fieldset> et donc de structurer le formulaire en fonction des champs qu'il contient, ce qui améliore la présentation. L'élément <legend> contient le titre de chacun de ces blocs. À l'intérieur de chaque bloc se trouvent les éléments XHTML qui créent les champs visibles ou invisibles du formulaire.

Les sections qui suivent rappellent les différents composants XHTML d'un formulaire et leurs rôles respectifs.

L'élément <input />

La balise unique <input /> permet de créer les composants classiques des formulaires que vous connaissez déjà et dont les aspects et les rôles sont très différents. La différenciation de ces composants s'effectue simplement en définissant la valeur de leurs attributs, et notamment de l'attribut type.

L'attribut name est obligatoire, car c'est lui qui permet d'identifier les champs côté serveur et ainsi de récupérer leur contenu. Les sections qui suivent détaillent les éléments de type "text", "password", "checkbox", "radio", "submit", "reset", "file" et "hidden".

L'élément *<input type="text" />*

Cet élément crée un champ de saisie de texte d'une seule ligne. En plus de l'attribut name, vous pouvez apporter des précisions supplémentaires à l'aide des attributs suivants :

- size="nombre". Détermine la largeur de la zone en nombre de caractères.
- maxlength="nombre". Détermine le nombre maximal de caractères que l'utilisateur est autorisé à saisir.
- value="texte". Définit un texte par défaut tant que l'utilisateur ne l'a pas modifié. C'est cette valeur qui est transmise au serveur si l'internaute ne saisit aucun texte, comme dans l'exemple ci-dessous :

```
<input type ="text" name="ville" size="30" maxlength="40" value="Votre ville"/>
```

L'aspect visuel de ce composant est illustré à la figure 6-1 (repères Ⓙ, @, Ⓢ et Ⓐ).

L'attribut *value*

Pour des raisons d'ergonomie, il est préférable que le texte par défaut défini à l'aide de l'attribut value s'efface tout seul au moment où l'utilisateur clique dessus car cela lui évite de devoir l'effacer.

Il suffit pour cela d'utiliser une instruction JavaScript très simple :

Pour réagir à l'événement clic :

```
<input type ="text" name="ville" size="30" maxlength="40" value="Votre ville"
onclick="this.value=" "/>
```

Pour que le texte s'efface dès que la zone de texte reçoit le focus (le curseur passe dans la zone) :

```
<input type ="text" name="ville" size="30" maxlength="40" value="Votre ville"
onfocus="this.value=" "/>
```

L'élément *<input type="password" />*

Crée un champ de saisie de mot de passe semblable à un champ texte mais dans lequel les caractères saisis sont invisibles et remplacés par des astérisques.

Les attributs size et maxlength y jouent le même rôle que précédemment.

En voici un exemple :

```
<input type ="password" name="code" size="10" maxlength="6"/>
```

L'élément *<input type="checkbox" />*

Crée une case à cocher utilisée pour effectuer un ou plusieurs choix parmi ceux qui sont préétablis par le programmeur. L'attribut value contient le texte qui sera transmis au serveur si l'utilisateur coche la case. Il est obligatoire.

Dans les groupes de cases à cocher, il est possible de cocher plusieurs cases simultanément, ce qui n'est pas le cas des cases d'option (boutons radio). Dans ce cas, il faut que le nom de tous les composants soit le même et qu'il soit suivi de crochets ouvrants et fermants de façon à récupérer les valeurs dans un tableau, comme vous le verrez en détail par la suite.

En voici un exemple :

```
<input type ="checkbox" name="lang[]" value="français" />
<input type ="checkbox" name="lang[]" value="anglais" />
```

L'aspect visuel de ce composant est illustré à la figure 6-1 (repère Ⓖ).

L'élément *<input type="radio" />*

Crée un bouton radio. Employé seul, un bouton radio peut servir à valider des choix. Employé en groupe, il implique, à la différence de cases à cocher, qu'un seul choix est autorisé. Dans ce cas, tous les boutons radio du groupe doivent avoir une même valeur pour l'attribut "name". Le fait d'en activer un désactive celui qui l'était auparavant.

L'attribut checked="checked" définit le bouton coché par défaut. L'attribut value joue le même rôle que pour les cases à cocher et est également indispensable.

Voici un exemple d'utilisation de l'élément "radio" :

```
<label>Débutant</label><input type ="radio" name="capa" value="débutant" />
<label>Initié</label><input type ="radio" name="capa" value="initié" />
```

L'aspect visuel de ce composant est illustré à la figure 6-1 (repère @).

L'élément *<input type="submit" />*

Crée un bouton sur lequel l'utilisateur doit cliquer pour déclencher l'envoi des données de tout le formulaire vers le serveur.

Il est indispensable d'avoir au moins un bouton d'envoi par formulaire, mais il est possible d'en utiliser plusieurs. Le clic sur l'un de ces boutons est alors analysé par le script désigné par l'attribut action de l'élément <form>. Cela permet d'effectuer des tâches spécialisées en fonction de la valeur associée à chaque bouton grâce à son attribut value. C'est le contenu de l'attribut value qui constitue le texte visible du bouton dans le formulaire.

Vous pourrez voir des exemples d'utilisation de plusieurs boutons d'envoi à la fin de ce chapitre ainsi qu'au chapitre 13. L'attribut name n'est *a priori* pas utile, en particulier s'il n'y a qu'un seul bouton d'envoi.

Voici un exemple simple d'utilisation de l'élément "submit" :

```
<input type ="submit" value="Envoyer" />
```

L'aspect visuel de ce composant est illustré à la figure 6-1 (repère ❾).

L'élément *<input type="reset" />*

Crée un bouton de réinitialisation du formulaire et non d'effacement systématique, comme on le croit souvent. Si les éléments du formulaire ont des attributs value qui définissent des valeurs par défaut, ce sont ces valeurs qui apparaissent au démarrage de la page et qui sont réaffichées si l'utilisateur clique sur le bouton reset.

Le contenu de l'attribut value du bouton d'effacement constitue le texte visible du bouton dans le formulaire.

En voici un exemple :

```
<input type ="reset" value="Effacer" />
```

L'aspect visuel de ce composant est illustré à la figure 6-1 (repère ❿).

L'élément *<input type="file" />*

Permet le transfert de fichiers du poste client vers le serveur. Cet élément crée un champ de saisie de même aspect qu'un champ de texte et un bouton de sélection de fichier permettant à l'utilisateur de choisir le fichier à transférer.

L'attribut name est obligatoire. Vous pouvez utiliser en plus les attributs size limitant la taille de la zone de saisie, et plus particulièrement l'attribut accept, qui définit le ou les types de fichier acceptés en transfert. Par sécurité, l'utilisation de cet attribut est recommandée car il permet de limiter le transfert à certains types de fichiers bien précis et de refuser les autres.

Dans l'exemple ci-dessous, le serveur n'accepte que le transfert de fichiers ayant les extensions .gif ou .jpeg et refuse tout autre type :

```
<input type="file" name="monfichier" accept="image/gif,image/jpeg" size="30"/>
```

L'aspect visuel de ce composant est illustré à la figure 6-1 (repère ⓟ).

L'élément *<input type="hidden" />*

Crée un champ caché n'ayant, comme son nom l'indique, aucun rendu visuel dans le formulaire mais permettant de transmettre des informations invisibles pour l'utilisateur.

Les informations sont contenues dans l'attribut value. L'attribut name est obligatoire.

En voici un exemple :

```
<input type="hidden" name="MAX_FILE_SIZE" value="20000"/>
```

L'élément <textarea>

À l'instar de l'élément <input /> avec l'attribut type=text, l'élément <textarea> crée un champ de saisie de texte mais, contrairement au précédent, en permettant la saisie sur plusieurs lignes.

Cet élément comporte une balise de fermeture </textarea> et un contenu de texte par défaut affiché dans la zone de texte. Les attributs cols et rows, qui donnent respectivement le nombre de colonnes et de lignes de la zone de texte, doivent être définis.

L'exemple suivant :

```
<textarea name="commentaire" cols="45" rows="8" >
Tapez vos commentaires ici
</textarea>
```

crée une zone de saisie de texte de 45 colonnes et 8 lignes au maximum.

L'aspect visuel de ce composant est illustré à la figure 6-1 (repère).

L'élément <select>

Crée une liste de sélection d'options parmi lesquelles l'utilisateur fait un choix, chaque option devant être définie par un élément <option> séparé.

L'élément <select> a la structure suivante :

```
<select name="maliste">
 <option value="valeur 1"> Texte choix 1</option>
  <option value="valeur 2"> Texte choix 2</option>
  <option value="valeur 3"> Texte choix 3</option>
</select>
```

Il comporte les attributs suivants :

- name="nom_select". Obligatoire. Donne le nom de la liste.
- size="Nombre". Détermine le nombre de choix visibles simultanément. Par défaut, sa valeur est 1.
- multiple="multiple". Autorise la sélection de plusieurs options simultanément.

L'élément <option> comporte les attributs suivants :

- value="chaine". Obligatoire. Définit la valeur transmise au serveur si l'option est sélectionnée.
- selected="selected". Définit l'option qui est sélectionnée par défaut dans la liste si l'utilisateur ne fait pas de choix.

L'aspect visuel de ce composant est illustré à la figure 6-1 (repère Ⓐ).

Exemple de code <form>

L'exemple 6-1 fournit le code d'un formulaire contenant tous les éléments possibles répartis en trois groupes :

- saisie des données personnelles du visiteur et zones de texte « nom », « prenom » et « mail » (repères Ⓙ, @ et Ⓢ), champ de saisie de mot de passe (repère Ⓐ), boutons radio (repère Ⓢ) et de sélection (repère Ⓐ) à quatre options ;
- cases à cocher (repère Ⓖ) et zone de texte multiligne (repère) ;
- envoi d'un fichier du client vers le serveur (repère Ⓟ), champ caché contenant la taille maximale des fichiers transférables et boutons de réinitialisation (repère ⓾) et d'envoi (repère $).

☞ **Exemple 6-1. Formulaire type**

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Formulaire traité par PHP</title>
<style type="text/css">
fieldset {border-width: 3px; border-style: double; border: #993300}
</style>
</head>
<body>
<form action="cible.php" method="post" enctype="application/x-www-form-urlencoded">
<!— Premier groupe de composants—>
  <fieldset>
    <legend><b>Vos coordonnées</b></legend>
    <label>Nom : </label><input type="text" name="nom" size="40" maxlength="256"
    ➥value="votre nom" /><br /> ←Ⓙ
    <label>Prénom : </label><input type="text" name="prenom" size="40"
    ➥maxlength="256" value="votre prénom" /><br /> ←@
    <label>Mail : </label><input type="text" name="mail" size="40" maxlength="256"
    ➥value="votre mail" /><br /> ←Ⓢ
    <label>Code : </label><input type="password" name="code" size="40"
    ➥maxlength="6" /> <br /> ←Ⓐ
    <input type="radio" name="sexe" value="homme" /> Homme<br /> ←@
    <input type="radio" name="sexe" value="femme" /> Femme<br />
    <select name="pays" size="1"> ←Ⓐ
    <option value="France"> France</option>
    <option value="Belgique"> Belgique</option>
    <option value="Suisse"> Suisse</option>
    <option value="Canada"> Canada</option>
    </select>
    <br />
  </fieldset>
<!— Deuxième groupe de composants—>
  <fieldset>
    <legend><b>Vos goûts</b></legend>
  <input type="checkbox" name="pomme" value="pomme" /> Pommes<br /> ←Ⓖ
   <input type="checkbox" name="poire" value="poire" /> Poires<br />
  <input type="checkbox" name="scoubidou" value="scoubidou" /> Scoubidous<br />
    <textarea name="gouts" cols="50" rows="5" onclick="this.value=''"> ←
     Décrivez vos goûts en détail
    </textarea>
    <br /><br />
  </fieldset>
<!— Troisième groupe de composants—>
  <fieldset>
    <legend><b>Envoyez nous votre photo</b></legend>
    <input type="file" name="fichier" accept="image/jpeg"  /> ←Ⓟ
    <input type="hidden" name="MAX_FILE_SIZE" value="10000" />
    <br /><br />
    <input type="reset" value="Effacer" /> ← ⓾
```

```
       <input type="submit" value="Envoyer" /> ←$
    <br /><br />
  </fieldset>
</form>
</body>
</html>
```

La figure 6-1 fournit une image du formulaire que vous venez de créer avec les repères correspondant au code XHTML.

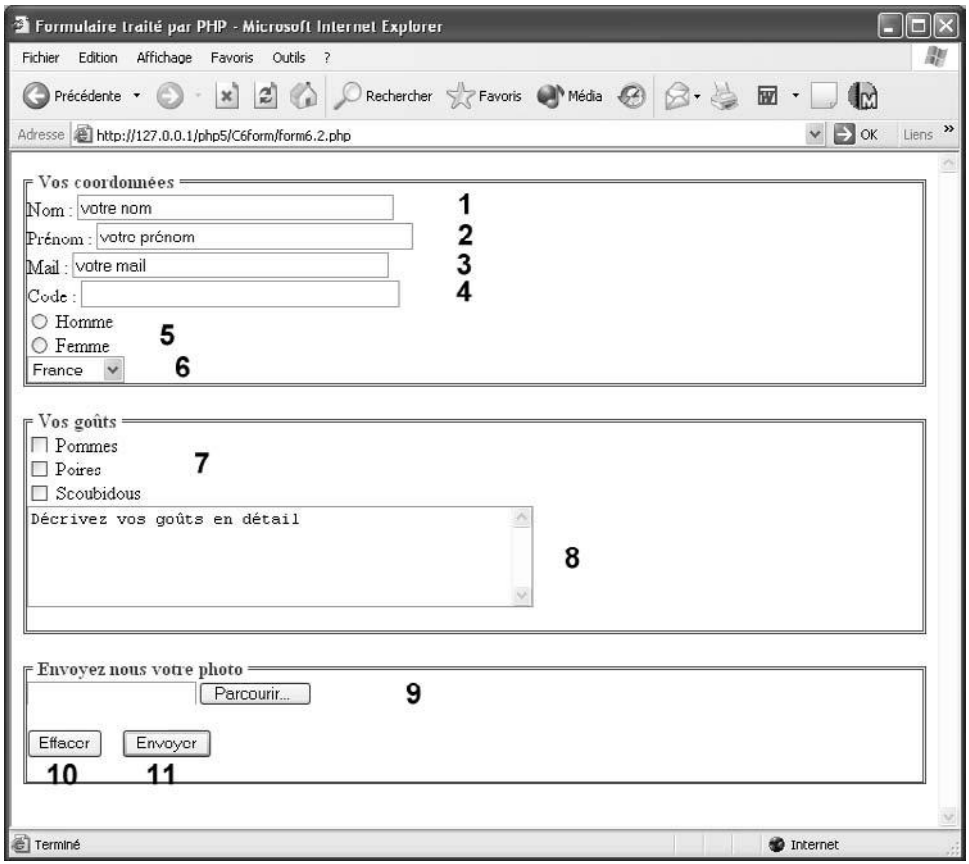


Figure 6-1 : *Formulaire type complet*

Fichier XHTML ou PHP ?

Le fichier peut être enregistré sans problème avec l'extension .html ou .htm, car il ne contient aucun code PHP. C'est le fichier cible.php, donné comme valeur de l'attribut action de l'élément <form>, qui traite les données du formulaire coté serveur. Il n'est pas gênant de l'enregistrer avec l'extension .php, même s'il ne contient aucun code PHP.

Récupération des données du formulaire

Maintenant que vous savez créer de beaux formulaires, vous allez voir comment récupérer les données entrées par l'utilisateur dans les différents champs du formulaire.

Tout d'abord, que se passe-t-il lorsque l'utilisateur clique sur le bouton d'envoi ? Une requête HTTP est envoyée au serveur à destination du script désigné par l'attribut action de l'élément <form>. La requête contient toutes les associations entre les noms des champs et leur valeur. Ces associations se trouvent dans l'en-tête HTTP si la méthode POST est utilisée et dans l'URL s'il s'agit de la méthode GET.

Valeurs uniques

Les valeurs uniques proviennent des champs de formulaire dans lesquels l'utilisateur ne peut entrer qu'une valeur, un texte par exemple, ou ne peut faire qu'un seul choix (bouton radio, liste de sélection à choix unique).

Depuis PHP 4.1, ces valeurs sont contenues sur le serveur dans des tableaux associatifs dits superglobaux appelés $_POST et $_GET, selon la méthode choisie. Les clés de ces tableaux sont les noms associés aux champs par l'attribut name.

Voyons tout de suite ce mécanisme à l'œuvre avec un formulaire élémentaire ne contenant qu'un seul champ de saisie de texte. Dans la suite de l'ouvrage, nous privilégions l'emploi de la méthode post pour l'envoi des formulaires. Dans ce chapitre, en revanche, nous envisageons la méthode get à titre informatif.

W3C

Le W3C, l'organisme de « normalisation » du Web, envisage de supprimer la valeur get pour l'attribut method des formulaires en HTML ou XHTML.

Cas de la méthode *POST*

Dans le code de l'exemple 6-2, l'élément <form> est écrit de la façon suivante (repère Ⓙ) :

```
action= "<?= $_SERVER["PHP_SELF"] ?>
```

Cela désigne le script lui-même comme cible pour le traitement des données.

Le code PHP proprement dit est le suivant (repère @) :

```
<?php
if(isset($_POST["nom"]) && isset($_POST["niveau"]))
{
echo "<h2> Bonjour ". stripslashes($_POST["nom"]). " vous êtes ".$_POST["niveau"].
➥" en PHP</h2>";
```

```
}
?>
```

Il contrôle d'abord l'existence des variables $_POST["nom"] et $_POST["niveau"], qui représentent respectivement le texte saisi et la valeur associée à la case cochée de façon à n'afficher le message qu'après l'envoi des données (repère Ⓢ). Sans ces précautions, le message s'afficherait même en l'absence de saisie de données par le visiteur. Ces variables n'existent que si l'utilisateur a complété les champs et a cliqué sur le bouton d'envoi. Lorsqu'elles existent, elles sont utilisées pour créer un affichage de bienvenue, comme illustré à la figure 6-2.

La fonction stripslashes() est utilisée pour supprimer les caractères d'échappement ajoutés automatiquement devant les caractères spéciaux éventuellement utilisés dans les données saisies avant de les afficher dans la page.

☞ Exemple 6-2. Récupération des valeurs dans un formulaire élémentaire

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
     "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Formulaire traité par PHP</title>
</head>
<body>
<form action= "<?= $_SERVER["PHP_SELF"] ?>" method="post"
➥ enctype="application/x-www-form-urlencoded"> ←Ⓙ
 <fieldset>
  <legend><b>Infos</b></legend>
<div>

Nom : <input type="text" name="nom" size="40" />
<br />
Débutant : <input type="radio" name="niveau" value="débutant" />
Initié :   <input type="radio" name="niveau" value="initié" /><br />
<input type="reset" value="Effacer" />
<input type="submit" value="Envoyer" />
</div>
</fieldset>
</form>
<?php ←@
if(isset($_POST["nom"]) && isset($_POST["niveau"])) ←Ⓢ
{
  echo "<h2> Bonjour ". stripslashes($_POST["nom"]). " vous êtes ".
  ➥$_POST["niveau"]." en PHP</h2>";
}
?>
```

```
</body>
</html>
```

Après avoir saisi « Jan Geelsen » dans le champ texte et coché la case « débutant », vous obtenez l'affichage illustré à la figure 6-2. Remarquez que les saisies ne sont plus visibles car la page a été entièrement réaffichée dans le navigateur ; pour conserver les données saisies dans le formulaire, consultez la section « Maintien de l'état du formulaire » plus loin dans ce chapitre.



Figure 6-2

Formulaire élémentaire et résultat

Cas de la méthode *GET*

Avec la méthode GET, vous récupérez les données du formulaire dans les variables $_GET ["nom"] et $_GET["niveau"], comme ci-dessous :

```
<?php
if(isset($_GET["nom"]) && isset($_GET["niveau"]))
{
  echo "<h2> Bonjour ". stripslashes ($_GET["nom"]). " vous êtes ".$_GET["niveau"].
  ➥" en PHP</h2>";
}
?>
```

Contrairement à ce qui se passe avec la méthode POST, vous constatez que lors du clic sur le bouton d'envoi l'adresse de la page cible désignée par l'attribut action est suivie par le caractère ? puis par le nom de chaque champ et la valeur qui y est associée.

Par exemple, si le nom de l'utilisateur est « Jean-René d'Orléans » et que la case «débutant » soit cochée, le navigateur affiche l'adresse complète :

```
http://localhost/php5/C6form/form6.3_get.php?nom=Jean-Ren%E9+d%92Orl%E9ans&niveau
➥=initi%E9
```

Les caractères accentués « é » et l'apostrophe sont codés en hexadécimal respectivement par %E9 et %92. Les espaces sont remplacées par le signe +, et chaque paire nom=valeur est séparée par le signe &.

Dans les versions antérieures à PHP 4.1, les données étaient récupérables dans les variables $HTTP_POST_VARS ou $HTTP_GET_VARS ou encore directement dans des variables globales portant le nom des champs du formulaire (la valeur de leur attribut name), soit ici $nom et $niveau. Ces méthodes sont à considérer comme obsolètes.

Pour simplifier la manipulation des valeurs issues du formulaire, vous pouvez récupérer chaque valeur dans des variables scalaires dès le début du script de traitement, comme ci-dessous :

```
$sonnom= $_POST["nom"];
$sonniveau=$_POST["niveau"];
```

puis les réutiliser sous cette forme par la suite.

Maintien de l'état du formulaire

Lorsque le script contenant le formulaire est chargé du traitement des données, l'ensemble de la page est réaffiché après traitement, de même que l'ensemble du formulaire. Le formulaire se retrouve alors dans son état initial, et toutes les saisies effectuées sont effacées (voir figure 6-2).



Figure 6-3 : *Maintien de l'état du formulaire*

En cas d'erreur de saisie sur un seul champ, l'utilisateur est obligé de recommencer l'ensemble de la saisie. S'il s'agit d'un long questionnaire, il y a de quoi s'impatienter. Pour éviter cela, il est utile de maintenir l'état du formulaire après traitement en réaffichant l'ensemble des saisies. Vous allez effectuer cette opération à partir du code de l'exemple 6-2 :

- Pour la zone de saisie de texte dont l'attribut name a la valeur "nom", il suffit de définir l'attribut value avec la variable $_POST["nom"], non sans avoir au préalable contrôlé l'existence de cette variable. Lors du premier affichage de la page, la zone est donc vide ou contient le dernier nom saisi (repère Ⓙ). Le code PHP est donc :

```
<?php if(isset($_POST["nom"])) echo $_POST["nom"]?>
```

- Pour les boutons radio dont l'attribut name a la valeur "niveau" et qui permettent le choix entre les valeurs "Débutant" et "Initié", il faut définir l'attribut checked du bouton choisi à la valeur "checked" en fonction de la valeur de la variable $_POST ["niveau"] (repère @). Pour le premier bouton, le code PHP devient :

```
<?php if(isset($_POST["niveau"]) && $_POST["niveau"]=="débutant")
➥echo "checked=\"checked\"" ?>
```

Après une saisie identique à celle de la figure 6-2, vous obtenez un écran identique à celui de la figure 6-3, dans lequel toutes les données saisies sont encore visibles après le traitement du formulaire.

☞ Exemple 6-3. Maintien de l'état du formulaire

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Formulaire traité par PHP</title>
</head>
<body>
<form action= "<?= $_SERVER["PHP_SELF"] ?>" method="post"
➥enctype="application/x-www-form-urlencoded">
<fieldset>
<legend><b>Infos</b></legend>
Nom : <input type="text" name="nom" size="40" value="<?php if(isset($_POST["nom"]))
➥echo $_POST["nom"]?>"/> ←Ⓙ
<br />
Débutant : <input type="radio" name="niveau" value="débutant"
<?php if(isset($_POST["niveau"]) && $_POST["niveau"]=="débutant")
➥echo "checked=\"checked\"" ?> /> ←@
Initié :    <input type="radio" name="niveau" value="initié"
<?php if(isset($_POST["niveau"]) && $_POST["niveau"]=="initié")
➥echo "checked=\"checked\"" ?>/><br />
<input type="reset" value="Effacer" />
<input type="submit" value="Envoyer" />
</fieldset>
</form>
<?php
if(isset($_POST["nom"]) && isset($_POST["niveau"]))
{
```

```
    echo "<h2> Bonjour ". stripslashes($_POST["nom"]). " vous êtes ".$_POST["niveau"].
    ➥" en PHP</h2>";
}
?>
</body>
</html>
```

Exemple pratique

L'exemple 6-4 offre une illustration pratique d'utilisation de formulaire. Il s'agit d'un site d'annonces immobilières proposant un plan de financement aux visiteurs.

L'application est constituée de deux fichiers, form4.html et form4.php**.**

Le fichier form4.html affiche le formulaire de saisie des données nécessaires au calcul du prêt (voir figure 6-4). Il ne contient aucun code PHP et peut donc être enregistré avec l'extension .html**.** L'attribut action de l'élément <form> (repère Ⓙ) désigne le fichier form4.php, qui est chargé du traitement des données et de l'affichage des résultats.

Le fichier form4.php vérifie d'abord l'existence des variables $_POST["capital"], $_POST["taux"] et $_POST["duree"], toutes nécessaires au calcul du prêt. La variable $_POST["assur"] est nécessaire dans tous les cas. Elle a la valeur 1 puisque le bouton radio « OUI » est coché par défaut. $capital correspond au capital emprunté (repère @). $taux désigne le taux mensuel sous forme décimale. Si l'utilisateur saisit 6 pour le taux annuel, la variable $taux vaut 6/100/12, soit 0,005, ou 0,5 % par mois (repère Ⓢ). $duree est la durée en mois (repère Ⓐ). $assur renvoie au montant de l'assurance mensuelle, soit 0,035 % du capital emprunté. Cette variable prend la valeur 0 si $_POST["assur"] vaut 0 (repère @).

Vient ensuite le calcul de la mensualité selon la formule financière suivante (repère Ⓐ) :

```
$mens=($capital*$taux)/(1-pow((1+$taux),–$duree))
```

Le script affiche la mensualité hors assurance (repère Ⓖ) ainsi que le tableau d'amortissement contenant, entre autres, le capital restant dû et les intérêts de chaque période (repère). La figure 6-5 donne un exemple de résultat.

Si le formulaire est incomplet, l'instruction header()affiche à nouveau la page de saisie form4.html**,** obligeant l'utilisateur à effectuer une saisie complète (repère Ⓟ).

☞ Exemple 6-4. Calcul de prêt bancaire

Page de saisie des données (fichier form4.html) :

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Calcul de prêts</title>
```

```
</head>
<body>
<h3>Prêts </h3>
<form method="post" action="form4.php"> ←Ⓙ
<fieldset>
<legend>Les caractéristiques de votre prêt</legend>
<table>
<tr>
<td>Montant</td>
<td><input type="text" name="capital" /></td>
</tr>
<tr>
<td>Taux</td>
<td><input type="text" name="taux" /></td>
</tr>
<tr>
<td>Durée en année</td>
<td><input type="text" name="duree" /></td>
</tr>
<tr>
<td>Assurance</td>
<td>OUI : <input type="radio" name="assur" checked="checked" value="1" /> 
➥NON<input type="radio" name="assur" value="0" /></td>
</tr>
<tr>
<td><input type="reset" name="" value="Effacer"/></td>
<td><input type="submit" name="" value="Calculer"/></td>
</tr>
</table>
</fieldset>
</form>
</body>
</html>
```

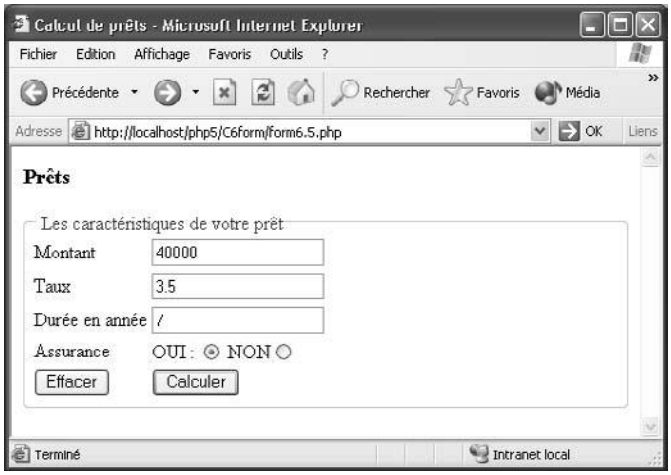


Figure 6-4

Page principale de saisie des données

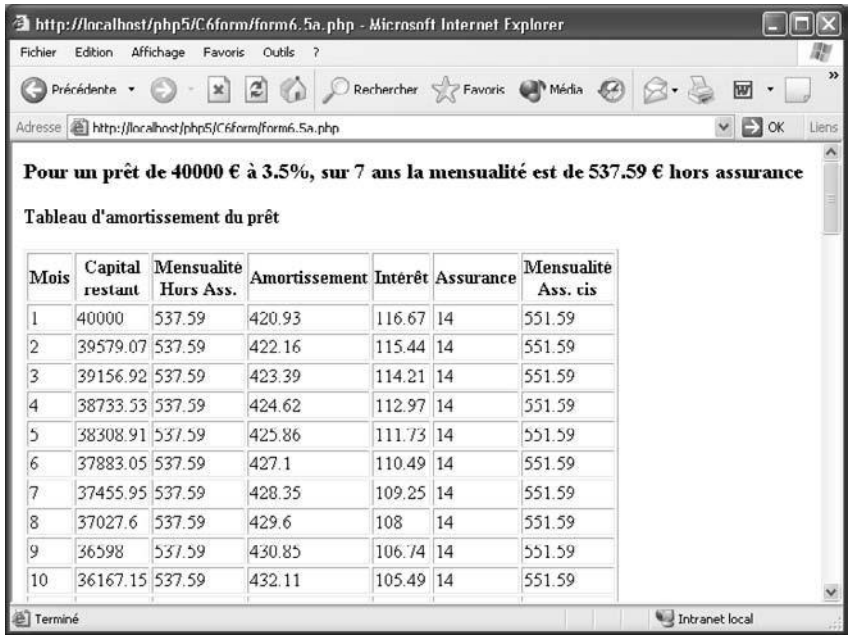
Page de traitement des données et d'affichage des résultats (fichier form4.php) :

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Tableau d'amortissement</title>
</head>
<body>
<div>

<?php
if(isset($_POST["capital"])&&$_POST["taux"]&&$_POST["duree"])
{
  $capital=$_POST["capital"]; ←@
  $taux=$_POST["taux"]/100/12; ←Ⓢ
  $duree=$_POST["duree"]*12; ←Ⓐ
  $assur=$_POST["assur"]*$capital*0.00035; ←@
  $mens=($capital*$taux)/(1-pow((1+$taux),–$duree)); ←Ⓐ
  echo "<h3>Pour un prêt de $capital &euro; à ", $_POST["taux"] ,"%, sur ",
  ➥$_POST["duree"]," ans la mensualité est de ",round($mens,2),"
  ➥&euro; hors assurance</h3>"; ←Ⓖ
  echo "<h4>Tableau d'amortissement du prêt</h4>";
  echo "<table border=\"1\"> <tr><th>Mois </th><th>Capital restant</ th><th>
  ➥Mensualité Hors Ass.</th><th>Amortissement </ th><th> Intérêt</th><th>
  ➥Assurance</th><th>Mensualité Ass.cis </ th>"; ←
//Boucle d'affichage des lignes du tableau
  for($i=1;$i<=$duree;$i++)
  {
    $int=$capital*$taux;
    $amort=$mens-$int;
    echo "<tr>";
    echo "<td>$i</td>";
    echo "<td>",round($capital,2),"</td>";
    echo "<td>",round($mens,2),"</td>";
    echo "<td>",round($amort,2),"</td>";
    echo   "<td>",round($int,2),"</td>";
    echo "<td>$assur</td>";
    echo "<td>",round($mens+$assur,2),"</td>";
    echo "</tr>";
    $capital=$capital-$amort;
  }
  echo "</table>";
}
else
{
  header("Location:form4.php"); ←ⓟ
}
?>
</div>
```

```
</body>
</html>
```



Mois	Capital restant	Mensualité Hors Ass.	Amortissement	Intérêt	Assurance	Mensualité Ass. cis
1	40000	537.59	420.93	116.67	14	551.59
2	39579.07	537.59	422.16	115.44	14	551.59
3	39156.92	537.59	423.39	114.21	14	551.59
4	38733.53	537.59	424.62	112.97	14	551.59
5	38308.91	537.59	425.86	111.73	14	551.59
6	37883.05	537.59	427.1	110.49	14	551.59
7	37455.95	537.59	428.35	109.25	14	551.59
8	37027.6	537.59	429.6	108	14	551.59
9	36598	537.59	430.85	106.74	14	551.59
10	36167.15	537.59	432.11	105.49	14	551.59

Figure 6-5
Page d'affichage des résultats

Les valeurs multiples

Certains champs de formulaire peuvent permettre aux visiteurs de saisir plusieurs valeurs sous un même nom de composant.

Cela peut concerner un groupe de cases à cocher ayant le même attribut name, par exemple, dont il est possible de cocher une ou plusieurs cases simultanément. Ce peut également être le cas d'une liste de sélection ayant toujours un nom unique mais dans laquelle l'attribut multiple="multiple" est défini. Il est enfin possible de donner le même nom à des éléments de saisie de texte différents, mais cela présente moins d'intérêt.

Dans tous les cas, ce n'est pas une valeur scalaire mais un tableau qui est récupéré côté serveur. Il faut pour cela faire suivre le nom du composant de crochets, comme pour créer une variable de type array.

Dans l'exemple suivant :

```
Bleu:<input type="checkbox" name="choix[]" value="bleu" />
Blanc:<input type="checkbox" name="choix[]" value="blanc" />
```

l'utilisateur peut cocher les deux cases simultanément. Le programmeur récupère ces

valeurs dans les variables suivantes :

```
$_POST["choix"][0] qui contient la valeur "bleu"
$_POST["choix"][1] qui contient la valeur "blanc"
```

La variable $_POST est un tableau multidimensionnel, en l'occurrence à deux dimensions.

L'exemple 6-5 illustre la méthode de récupération des valeurs multiples. Le formulaire créé par le fichier form5.html contient trois zones de saisie de texte portant le même nom, une liste de sélection avec l'attribut multiple et quatre cases à cocher ayant le même nom. Dans les listes de sélection, l'utilisateur doit maintenir la touche Ctrl enfoncée pour faire plusieurs choix. Il peut être utile de lui rappeler cette fonctionnalité.

L'objet du formulaire est de faire saisir une fiche de renseignements par l'utilisateur puis d'afficher l'ensemble de ces informations. Le script cible du formulaire contenu dans le fichier form5.php récupère les données et réalise une fiche récapitulative des renseignements personnels si les variables du tableau $_POST existent (repère Ⓙ) ou, dans le cas contraire, une boîte d'alerte, à l'aide de la fonction JavaScript alert() (repère @), et une redirection vers la page de saisie, *via* la fonction JavaScript window.history.back() (repère Ⓢ).

La figure 6-6 illustre le formulaire de saisie et la figure 6-7 un exemple de fiche de résultat.

☞ Exemple 6-5. Récupération des valeurs multiples

Le fichier form5.html **:**

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Listes à choix multiples</title>
</head>
<body>
<form method="post" action="form5.php" >
<fieldset>
<legend>Recherche d'emploi: complétez la fiche </legend>
<div>
<span>Nom<input type="text" name="ident[]" />
Prénom<input type="text" name="ident[]" />
Age<input type="text" name="ident[]" />
<br /><br />
Langues pratiquées<br />
<select name="lang[]" multiple="multiple">
<option value="français"> français</option>
<option value="anglais"> anglais</option>
<option value="allemand"> allemand</option>
<option value="espagnol"> espagnol</option>
</select><br /><br />
Compétences informatiques<br />
XHTML<input type="checkbox" name="competent[]" value="XHTML" />
```

```
PHP<input type="checkbox" name="competent[]" value="PHP" />
MySQL<input type="checkbox" name="competent[]" value="MySQL" />
ASP.Net<input type="checkbox" name="competent[]" value="ASP.Net" />
</span><br /><br />
<input type="reset" value="EFFACER"/>
<input type="submit" value="ENVOI"/>
</div>
</fieldset>
</form>
</body>
</html>
```

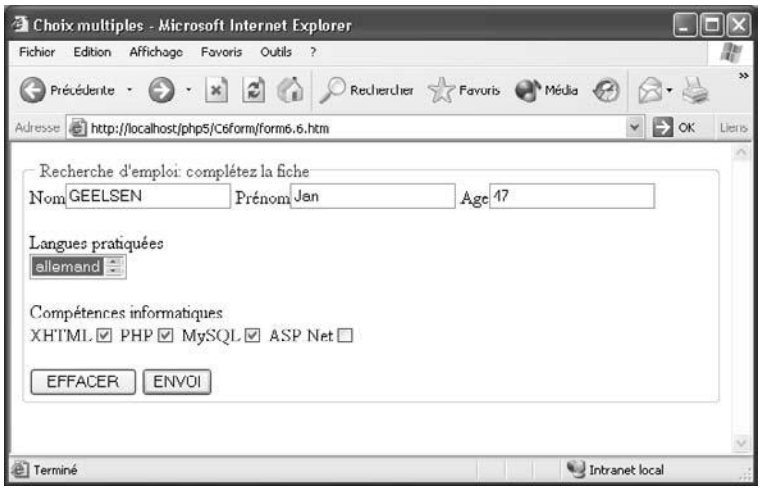


Figure 6-6

Formulaire de saisie

Le fichier form5.php **:**

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Compétences Info</title>
</head>
<body>
<?php
if(isset($_POST["ident"]) && isset($_POST["lang"]) &&
➥isset($_POST["competent"])) ←Ⓙ
{
echo "<table border=\"1\"><tr><th> Récapitulatif de votre fiche d'information
➥personnelle</th></tr><tr><td>";
$nom=$_POST["ident"][0];
```

```
$prenom=$_POST["ident"][1];
$age=$_POST["ident"][2];
$lang = $_POST["lang"];
$competent=$_POST["competent"];
echo"Vous êtes :<b> $prenom ", stripslashes($nom) ,"</b><br />Vous avez
➥<b>$age ans </b> ";
  echo "<br />Vous parlez :";
  echo "<ul>";
  foreach($lang as $valeur)
  {
  echo " <li> $valeur </li>";
  }
  echo "</ul>";
  echo "Vous avez des compétences informatiques en :";
  echo  "<ul>";
  foreach($competent as $valeur)
  {
    echo "<li> $valeur </li> ";
  }
  echo "</ul> </td></tr>";
  }
  else
  {
  echo"<script type=\"text/javascript\">" ;
  echo "alert('Cochez au moins une compétence!!');"; ←@
  echo "window.history.back();"; ←Ⓢ
  echo "</script>";
  }
  ?>
  </body>
  </html>
```



Figure 6-7 : *Page de résultat*

Transfert de fichier vers le serveur

L'inclusion d'un élément XHTML <input type="file" /> dans un formulaire crée une situation particulière. Il ne s'agit plus de transmettre une ou plusieurs valeurs scalaires au serveur mais l'intégralité d'un fichier, lequel peut avoir une taille importante et un type quelconque. Ce fichier doit évidemment être présent sur l'ordinateur du visiteur.

Par rapport au transfert de données, le transfert de fichiers présente un problème de sécurité pour votre site puisque des fichiers vont être écrits et éventuellement exécutés sur le serveur.

Supposez qu'un utilisateur mal intentionné transfère un fichier nommé index.php ou index.html sur le serveur. Ce fichier est exécuté automatiquement à l'appel de l'URL de votre nom de domaine. Pour éviter tout problème, il est prudent de prévoir une vérification de l'extension du fichier préalablement à la définition des extensions autorisées lors de la création du formulaire.

Pour cette définition, vous utilisez l'attribut accept de l'élément <input />. Cet attribut peut prendre un grand nombre de valeurs, correspondant aux types MIME des fichiers acceptés, par exemple "image/gif", comme dans l'exemple 6-6, ou "text/html". Un fichier d'un type différent est rejeté, ce qui confère une protection contre les utilisateurs mal intentionnés.

Contrairement aux exemples précédents, l'élément <form> doit avoir l'attribut method à la valeur post.

Précaution supplémentaire, vous pouvez envisager de limiter la taille des fichiers à télécharger en ajoutant au formulaire un champ caché nommé MAX_FILE_SIZE, dont l'attribut value contienne la taille maximale admise en octet. Cette valeur est récupérée dans la variable $_POST["MAX_FILE_SIZE"] (repère @) lorsque le champ caché est défini par le code suivant (repère Ⓙ) :

```
<input type="hidden" name="MAX_FILE_SIZE" value="100000" />
```

L'utilisation de ce champ n'est pas obligatoire puisque le fichier php.ini du serveur contient la directive "upload_max_filesize", dont la valeur est un entier indiquant la taille maximale en octet admise par défaut par le serveur. En local avec Wampserver, cette valeur est de 2 Mo. L'hébergeur peut toutefois définir une valeur très différente, qu'il vous appartient de vérifier à l'aide de la fonction phpinfo().

L'ajout d'un bouton d'envoi à votre formulaire est bien sûr toujours indispensable.

Lors de l'affichage de la page, l'utilisateur se retrouve face à ce qui ressemble à une zone de saisie de texte accompagnée d'un bouton dont l'intitulé est « Parcourir... ». Il peut saisir manuellement l'emplacement exact du fichier à transférer ou le choisir sur son disque en cliquant sur le bouton, comme il le ferait pour ouvrir un fichier à partir d'un traitement de texte, par exemple. Il n'est donc pas nécessaire de lui expliquer la procédure en détail.

Une fois le fichier sélectionné, un clic sur le bouton Submit provoque l'envoi du fichier au serveur et son traitement par un script, que vous devez encore écrire. À ce moment, le

fichier est bien transféré sur le serveur, mais il se trouve dans un répertoire tampon défini par la directive "upload_tmp_dir" du fichier php.ini**.** Si vous n'en faites rien, il est perdu lors de la déconnexion du client. De plus, il est enregistré sous un nom différent de celui qu'il avait sur le poste client, et vous ne savez pas encore sous quel nom, puisque celui-ci est créé arbitrairement par le serveur.

De même que pour le transfert de données simples, vous allez utiliser les tableaux superglobaux $_POST ou $_GET. Vous disposez également depuis la version PHP 4.1 du tableau associatif multidimensionnel $_FILES, qui contient les informations nécessaires au traitement du fichier transféré.

Si, comme dans le code de l'exemple 6-6, le nom de l'élément <input type="file".. /> est name="fich", vous pouvez lire les valeurs suivantes :

- $_FILES["fich"]["name"] : contient le nom qu'avait le fichier sur le poste client.
- $_FILES["fich"]["type"] : contient le type MIME initial du fichier et permet un contrôle et une censure éventuelle du fichier transféré.
- $_FILES["fich"]["size"] : donne la taille réelle en octet du fichier transféré.
- $_FILES["fich"]["tmp_name"] : donne le nom temporaire que le serveur attribue automatiquement au fichier en l'enregistrant dans le répertoire tampon.
- $_FILES["fich"]["error"] : donne le code d'erreur éventuel associé au fichier téléchargé et permet d'afficher un message d'erreur en clair en créant un tableau indicé de 0 à 4 contenant les messages appropriés. Ces codes sont définis par les constantes entières suivantes depuis PHP 4.3 :
  - UPLOAD_ERR_OK : de valeur 0, indique que le transfert est bien réalisé.
  - UPLOAD_ERR_INI_SIZE : de valeur 1, indique que la taille du fichier est supérieure à celle définie dans la php.ini**.**
  - UPLOAD_ERR_FORM_SIZE : de valeur 2, indique que la taille est supérieure à celle définie dans le champ caché MAX_FILE_SIZE.
  - UPLOAD_ERR_PARTIAL : de valeur 3, indique que le fichier n'a été que partiellement téléchargé.
  - UPLOAD_ERR_NO_FILE : de valeur 4, indique qu'aucun fichier n'a été téléchargé.

En dernier lieu, vous devez procéder à l'enregistrement et au renommage éventuel du fichier sur le serveur. Vous disposez pour cela de la fonction move_uploaded_file(), dont la syntaxe est la suivante :

```
boolean move_uploaded_file( string "fichtmp", string "fichfinal")
```

"fichtmp" est le chemin d'accès du fichier temporaire et "fichfinal" le nom sous lequel sera enregistré définitivement le fichier. La fonction retourne TRUE si l'opération est bien réalisée et FALSE dans le cas contraire.

Dans l'exemple 6-6, le code suivant (repère Ⓢ) :

move_uploaded_file($_FILES["fich"]["tmp_name"],"imagephp.gif");

enregistre le fichier transféré désigné par $_FILES["fich"]["tmp_name"] dans le répertoire du fichier form6.7.php du serveur sous le nom "imagephp.gif". Le fichier est alors pleinement utilisable dans une page du site, comme si vous l'aviez transféré vous-même à l'aide d'un logiciel FTP.

Si le transfert est bien réalisé, vous affichez un message de bonne fin (repère Ⓐ). En cas de problème de transmission ou de renommage du fichier, le script affiche le code de l'erreur générée à l'aide de la variable $_FILES["fich"]["error"] (repère @).

☞ Exemple 6-6. Transfert de fichier

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Transfert de fichier</title>
</head>
<body>
<form action="form6.php" method="post" enctype="multipart/form-data">
<fieldset>
<input type="hidden" name="MAX_FILE_SIZE" value="100000" /> ←Ⓙ
<legend><b>Transfert de fichier</b></legend>
<table>
<tbody>
<tr>
<th>Fichier</th>
<td> <input type="file" name="fich" accept="image/gif" size="50"/></td>
</tr>
<tr>
<th>Clic!</th>
<td> <input type="submit" value="Envoi" /></td>
</tr>
</tbody>
</table>
</fieldset>
</form>
<!—Code PHP —>
<?php
if(isset($_FILES['fich']))
{
   echo "Taille maximale autorisée :",$_POST["MAX_FILE_SIZE"],
   ➥" octets<hr / >"; ←@
   echo "<b>Clés et valeurs du tableau \$_FILES </b><br />";
foreach($_FILES["fich"] as $cle =>  $valeur)
{
   echo "clé : $cle  valeur : $valeur <br />";
```

```
}
//Enregistrement et renommage du fichier
$result=move_uploaded_file($_FILES["fich"]["tmp_name"],"imagephp.gif"); ←Ⓢ
if($result==TRUE)
  {echo "<hr /><big>Le transfert est réalisé !</big>";} ←Ⓐ
  else
     {echo "<hr /> Erreur de transfert n°",$_FILES["fich"]["error"];} ←@
  }
  ?>
  </body>
  </html>
```

La figure 6-8 illustre l'aspect de la page d'envoi des fichiers et la figure 6-9 celui de la page de confirmation affichant le résultat de l'opération de transfert et les clés et valeurs du tableau $_FILES.

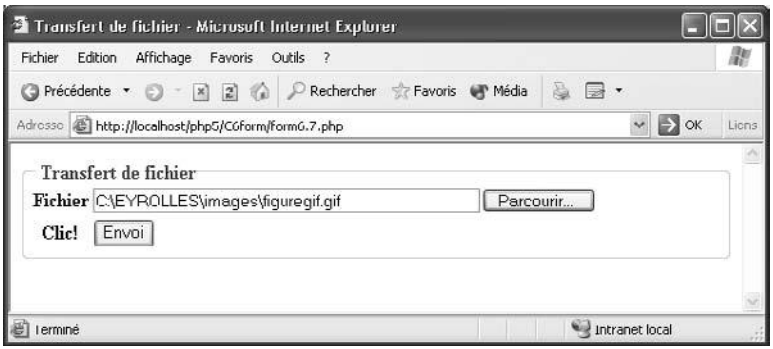


Figure 6-8 : *Formulaire d'envoi de fichier*

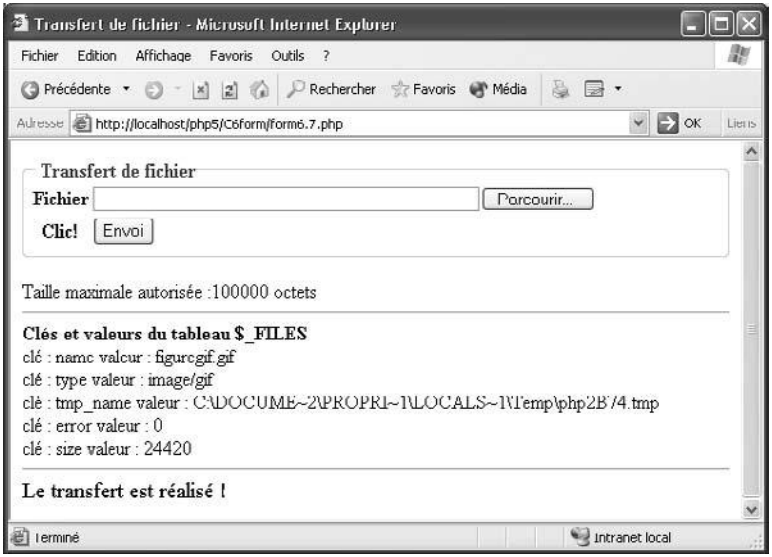


Figure 6-9 *Page de confirmation du transfert*

Transfert de plusieurs fichiers

Il est possible de proposer à l'utilisateur de transférer plusieurs fichiers simultanément pour lui éviter de recommencer plusieurs fois la même opération à partir de l'exemple précédent. Vous devez en ce cas faire en sorte que les éléments <input type="file"…/> soient tous écrits de la manière suivante :

```
<input type="file" name="fich[]"accept="image/gif" size="50"/>
```

avec le même nom suivi de crochets (repères Ⓙ et @ de l'exemple 6-7) ; l'attribut accept peut prendre des valeurs différentes pour chaque élément <input />.

La page de transfert correspondante est illustrée à la figure 6-10.

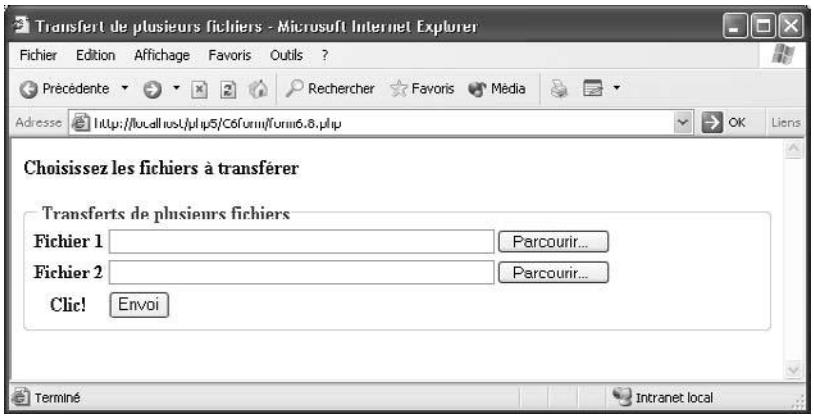


Figure 6-10 : *Formulaire de transfert de plusieurs fichiers*

Vous récupérez les informations sur les fichiers transférés dans la tableau $_FILES, devenu multidimensionnel à trois dimensions.

Par exemple, la variable suivante :

```
$_FILES["fich"]["name"][0]
```

permet de connaître le nom du premier fichier transféré.

Vous pouvez lire les caractéristiques de chacun des fichiers transférés à l'aide des boucles foreach imbriquées (repères Ⓢ et Ⓐ). Ce code n'a toutefois d'intérêt que pour le programmeur lors des tests et est à supprimer dans un site réel.

Vous procédez enfin au renommage et à la sauvegarde des fichiers transférés (repères @ et Ⓐ) puis au contrôle du bon déroulement de l'ensemble des opérations (repère Ⓖ).

☞ **Exemple 6-7. Transfert simultané de plusieurs fichiers**

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
      "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Transfert de plusieurs fichiers</title>
</head>
<body>
<!— Code HTML du formulaire —>
<form action="<?= $_SERVER['PHP_SELF'] ?>" method="post"
➥enctype="multipart/form-data" >
<fieldset>
<input type="hidden" name="MAX_FILE_SIZE" value="100000" />
<legend><b>Transferts de plusieurs fichiers</b></legend>
<table>
<tbody>
<tr>
<th>Fichier 1</th>
<td> <input type="file" name="fich[]" accept="image/gif" size="50"/></td> ←Ⓙ
</tr>
<tr>
<th>Fichier 2</th>
<td> <input type="file" name="fich[]" accept="image/gif" size="50"/> ←@
</td>
</tr>
<tr>
<th>Clic!</th>
<td> <input type="submit" value="Envoi" /></td>
</tr>
</tbody>
</table>
</fieldset>
</form>
<!— Code PHP —>
<?php
if(!empty($_FILES))
{
   echo "Taille maximale autorisée :",$_POST["MAX_FILE_SIZE"]," octets<hr / >";
   foreach($_FILES["fich"] as $cle => $valeur) ←Ⓢ
   {
      echo "Clé : $cle <br />";
      foreach($valeur as $key=>$val) ←Ⓐ
      {
         echo " Fichier : $key valeur : $val <br />";
      }
   }
```

```
//Déplacement et renommage des  fichiers
$result1=move_uploaded_file($_FILES["fich"]["tmp_name"][0],"image1.gif"); ←@
```

```
    $result2=move_uploaded_file($_FILES["fich"]["tmp_name"][1],"image2.gif"); ←Ⓐ
    if($result1==true && $result2==true)  ←Ⓖ
    {echo "<br /> Transferts réalisés
    !<br />";} else
    {echo "<br /> Transferts non effectués <br />";}
  }
  else echo "<h4>Choisissez les fichiers à transférer </h4>";
  ?>
  </body>
  </html>
```

Les informations écrites par les boucles foreach ont l'aspect suivant en cas de réussite des transferts :

```
Taille maximale autorisée :100000 octets

Clé : name
Fichier : 0 valeur :
figuregif.gif Fichier : 1
valeur : figuregif2.gif
Clé : type
Fichier : 0 valeur :
image/gif Fichier : 1
valeur : image/gif
Clé : tmp_name
Fichier : 0 valeur :
C:\DOCUME~2\PROPRI~1\LOCALS~1\Temp\php2B8C.tmp Fichier
: 1 valeur :
C:\DOCUME~2\PROPRI~1\LOCALS~1\Temp\php2B8D.tmp Clé :
error
Fichier : 0 valeur : 0
Fichier : 1
valeur : 0
Clé : size
Fichier : 0 valeur : 24420
Fichier : 1 valeur :

27614 Transferts
réalisés !
```

Gérer les boutons d'envoi multiples

L'utilisation de plusieurs boutons submit dans un même formulaire permet de déclencher des actions différentes en fonction du bouton activé par l'utilisateur. Il est nécessaire pour cela que les boutons aient le même nom et que la sélection de l'action se fasse en fonc- tion de la valeur associée à chaque bouton *via* l'attribut value.

L'exemple 6-8 crée une calculatrice en ligne proposant les opérations d'addition, de soustraction, de division et de puissance. Il est évidemment possible d'ajouter autant d'opérations que nécessaire. À chaque opération est associé un bouton submit (repères Ⓙ, @, Ⓢ et Ⓐ).

Deux zones de saisie de texte permettent d'entrer deux opérandes. Dans cet exemple, vous utilisez le maintien de l'état du formulaire de façon que ces derniers restent visibles lors de l'affichage du résultat (repères @ et Ⓐ).

Le code PHP commence comme d'habitude par vérifier l'existence des variables conte- nues dans le tableau $_POST en fonction de la valeur contenue dans la variable $_POST ["calcul"] de l'attribut value des boutons

Submit. Selon l'opération désirée, une instruction switch effectue le calcul approprié, dont le résultat est contenu dans la variable $resultat. Cette valeur est affichée dans le champ de text nommé result (repère Ⓖ).

☞ **Exemple 6-8. Une calculatrice en ligne**

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
     "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
<head>
     <title>Calculatrice en ligne</title>
     <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1" />
</head>
<body>
<!—Code PHP —>
<?php
if(isset($_POST["calcul"])&&isset($_POST["nb1"])&&isset($_POST["
nb2"]))
{
  switch($_POST["calcul"])
  {
  case "Addition x+y":
  $resultat=
  $_POST["nb1"]+$_POST["nb2"];
  break;
  case "Soustraction x-y":
  $resultat= $_POST["nb1"]-
  $_POST["nb2"]; break;
  case "Division x/y":
  $resultat=
  $_POST["nb1"]/$_POST["nb2"];
  break;
  case "Puissance x^y":
  $resultat=
  pow($_POST["nb1"],$_POST["nb2"]);
  break;
}
}
else
{
  echo "<h3>Entrez deux nombres : </h3>";
}
?>
<!— Code HTML du formulaire —>
<form action="<?=$_SERVER['PHP_SELF']?>" method="post" >
<fieldset>
<legend><b>Calculatrice en ligne</b></legend>
<table>
<tbody>
<tr>
<th>Nombre X</th>
<td> <input type="text" name="nb1" value="<?php if(isset($_POST["nb1"]))
➥echo $_POST['nb1'];else echo" ?>"/> ←@
</td>
</tr>
<tr>
  <th>Nombre Y</th>
<td> <input type="text" name="nb2" value="<?php if(isset($_POST["nb2"]))
➥echo $_POST['nb2'];else echo" ?>"/> ←Ⓐ
</td>
</tr>
<tr>
  <th>Résultat </th>
  <td> <input type="text" name="result" value="<?php if(isset($resultat))
➥echo $resultat;else echo"?>"/> ←Ⓖ
```

```
</td>
</tr>
<tr>
  <th>Choisissez!</th>
  <td>
    <input type="submit" name="calcul" value="Addition x+y" /> ←Ⓙ
    <input type="submit" name="calcul" value="Soustraction x-y" /> ←@
    <input type="submit" name="calcul" value="Division x/y" /> ←Ⓢ
    <input type="submit" name="calcul" value="Puissance x^y" /> ←Ⓐ
  </td>
</tr>
</tbody>
</table>
</fieldset>
</form>
</body>
</html>
```

La figure 6-11 illustre une page de résultat pour la fonction Puissance.

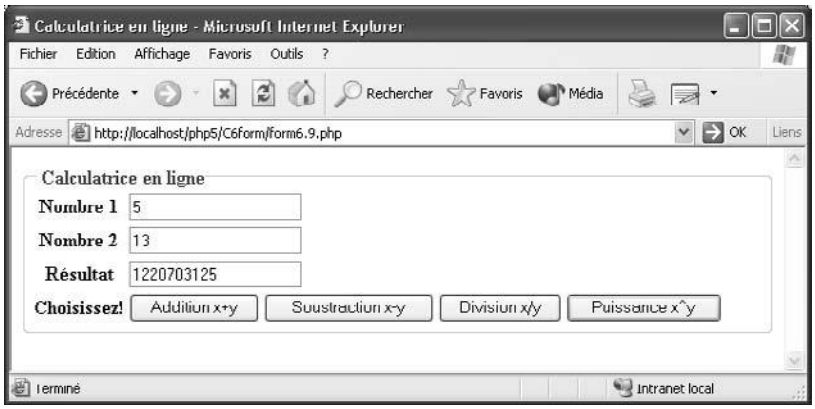


Figure 6-11

Une calculatrice en ligne